From Features to Machine Learning

Week 12 — Session 2: Training, evaluation & the limits of hand-crafted features

Practical Image Processing

Lecture 12.2 ▪ 60–70 min

Recap of Session 1

- We turn an image into a compact **feature vector** (HOG + optionally colour).
- HOG histograms gradient *orientations* per cell — a robust shape signature.
- A **classifier** (k-NN, SVM) learns a boundary in that feature space.
- We set up a **stratified** 70/30 train/test split.

Today

Train the classifiers, measure them *honestly*, and confront overfitting on look-alike cars.

Roadmap

Training the classifier

Evaluating honestly

Doing it right

In-class exercise

The limit of hand-crafted features

Wrap-up & deliverable

Training the classifier

The whole ML pipeline in code

```
X = [hog(gray(img)) for img in images]      # features
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
                                      stratify=y)
clf = SVC(kernel='rbf', C=10).fit(Xtr, ytr) # learn
print(accuracy_score(yte, clf.predict(Xte)))# HONEST score
print(confusion_matrix(yte, clf.predict(Xte)))
```

Extract features $\rightarrow$ split $\rightarrow$ fit $\rightarrow$ evaluate on unseen data. Always in that order.

Building X and y from your images

```python
from skimage.feature import hog
import cv2, numpy as np

def features(path):
    g = cv2.cvtColor(cv2.imread(path), cv2.COLOR_BGR2GRAY)
    g = cv2.resize(g, (96, 96))
    return hog(g, pixels_per_cell=(8,8), cells_per_block=(2,2),
               orientations=9)

X = np.array([features(p) for p in paths])   # (N, ~900)
y = np.array(labels)                         # 0=206,1=405,2=Dena
```

Every image becomes one row of X; `y` holds its class. Resize first so all vectors are the same length.

Fitting both classifiers

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline

knn = make_pipeline(StandardScaler(),
                    KNeighborsClassifier(n_neighbors=5))
svm = make_pipeline(StandardScaler(),
                    SVC(kernel='rbf', C=10))
knn.fit(Xtr, ytr);  svm.fit(Xtr, ytr)
```

✔ Good practice

Wrapping the scaler in a `Pipeline` means it is fit on training folds only — no leakage into the test set.

Evaluating honestly

Our real results (and an honest lesson)

Classifier	Train acc	Test acc
k-NN (k=5)	0.65	0.43
SVM (RBF)	1.00	0.48

Chance is 0.33 (3 classes). Both beat chance — but the SVM scores **100% on train, 48% on test**. That gap is **overfitting**.

Overfitting, explained

- **Overfitting**: the model memorizes training quirks instead of the real pattern.
- Symptom: train accuracy $\gg$ test accuracy (here 1.00 vs 0.48).
- Cures: more data, simpler model, regularization, better/invariant features.

Key idea

A model that is perfect on training data and mediocre on test data has learned the wrong thing.

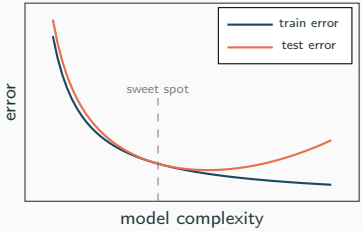
Why our SVM overfit so hard

- Only 126 training vectors, each 900-dimensional — far more features than examples.
- An RBF SVM with high C can bend the boundary to fit every training point exactly.
- Result: train 1.00, but the boundary is tuned to noise $\rightarrow$ test collapses to 0.48.

Key idea

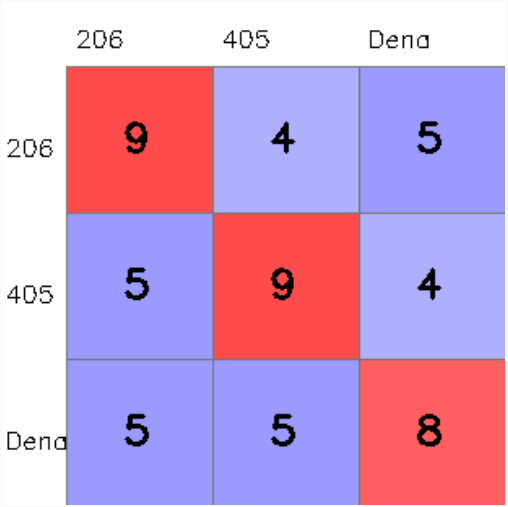
More dimensions than samples is a red flag: the model has enough freedom to memorize. Reduce C, reduce dimensions (PCA), or get more data.

The bias–variance tradeoff



- Too simple ⇒ high **bias**: underfits, both errors high.
- Too complex ⇒ high **variance**: overfits, test error climbs back up.
- Our RBF SVM sits far to the right: train error 0, test error large.

Reading the confusion matrix



- Rows = true class, columns = predicted.
- Diagonal = correct; off-diagonal = the mistakes.
- Shows *which* models get confused — far more useful than a single accuracy number.

Confusion matrix — working the numbers

A 3-class test set (rows = true, cols = predicted):

	206	405	Dena
206	**8**	3	1
405	4	**6**	2
Dena	2	3	**7**

For class **206**:

Math

$TP = 8,\ FP = 4+2 = 6,\ FN = 3+1 = 4.$

Precision $= \frac{8}{8+6} = 0.57.$

Recall $= \frac{8}{8+4} = 0.67.$

206 is most confused with 405 (3 and 4 off-diagonal both ways).

Metrics beyond accuracy

- **Accuracy** hides imbalance: 95% "all normal" is useless if 5% are the faults you need.
- **Precision**: of predicted-positive, how many were right? **Recall**: of actual-positive, how many did we catch?
- **F1**: their harmonic mean. Report these per class, not one global number.

Math

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1} = \frac{2\,PR}{P + R}.$$

Precision vs recall — a worked tradeoff

A fault detector on 100 parts (4 truly faulty):

- "Always OK" $\Rightarrow$ accuracy 96%, but recall on faults $= 0$. Useless.
- Flag anything suspicious $\Rightarrow$ recall $\uparrow$, precision $\downarrow$ (many false alarms).

Math

Catch 3/4 faults but raise 6 false alarms:

$$\text{Precision} = \frac{3}{3+6} = 0.33,$$

$$\text{Recall} = \frac{3}{4} = 0.75,$$

$$\text{F1} = \frac{2 \cdot 0.33 \cdot 0.75}{0.33 + 0.75} \approx 0.46.$$

Key idea

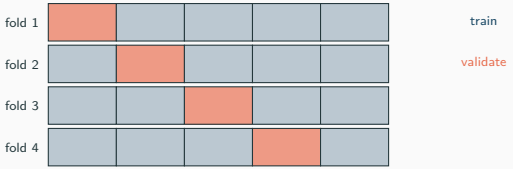
Which matters more depends on the cost: missing a fault vs raising a false alarm.

Doing it right

Cross-validation & not fooling yourself

- A single split is noisy. **k-fold cross-validation** averages over k splits for a stable estimate.
- Tune hyper-parameters on a **validation** set (or CV) — never on the test set.
- The test set is used *once*, at the very end. Peeking at it inflates your reported score.

k-fold cross-validation, illustrated



- Split data into k folds (here k=5, one row shown per fold used for validation).
- Each fold serves once as validation, $k-1$ times for training.
- Average the k scores $\Rightarrow$ a stable estimate with less luck.

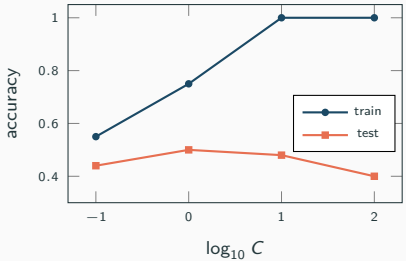
Hyper-parameter tuning with grid search

```python
from sklearn.model_selection import GridSearchCV
grid = {'svc__C': [0.1, 1, 10, 100],
        'svc__gamma': ['scale', 0.01, 0.001]}
gs = GridSearchCV(svm, grid, cv=5)    # 5-fold CV inside train
gs.fit(Xtr, ytr)
print(gs.best_params_)                # chosen WITHOUT the test set
print(gs.score(Xte, yte))             # test used ONCE, at the end
```

✕ Pitfall

Never put `Xte` inside `GridSearchCV`. Tuning on the test set is how honest projects quietly cheat.

Regularization: the C knob



- Large C: fit training hard $\Rightarrow$ train $\to 1.0$, test drops (overfit).
- Small C: softer margin, more errors allowed $\Rightarrow$ better generalization.
- Pick C by cross-validation, not by the train score.

✔ Good practice

The best test score here is at a *moderate* C, not the largest.

In-class exercise

In-class exercise (10 min)

Diagnose the model

1. Our SVM got train 1.00, test 0.48. Name the problem and two ways to reduce it.
2. A fault detector reports 96% accuracy but faults are 4% of data. Why might it be useless?
3. You want to tell 206 from 405. What feature besides HOG might help separate them?

Debrief

- (1) **Overfitting**: gather more data, simplify/regularize the SVM (lower C), or use better features.
- (2) Predicting "no fault" always scores 96% but has **zero recall** on faults — accuracy is the wrong metric.
- (3) **Colour histograms** (models differ in typical colour), size/aspect, or logo region features.

Key idea

A number without a train/test split and a confusion matrix tells you almost nothing.

The limit of hand-crafted features

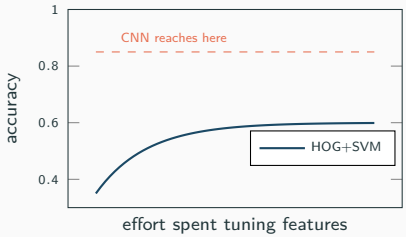
Feature engineering vs feature learning

- Today we **engineered** features (HOG) and learned only the classifier.
- The 48% ceiling on look-alike cars hints at the limit of hand-designed features.
- **Deep learning** (Weeks 13–14) *learns the features too*, directly from pixels.

Key idea

The classical pipeline plateaus on hard, fine-grained problems. That plateau is exactly why CNNs took over.

The plateau of hand-crafted features



- More feature tuning gives diminishing returns — the curve flattens.
- The gap to a learned representation stays open on fine-grained tasks.
- You cannot hand-design your way past look-alike classes.

Where this fits DocScan

- DocScan must read digits. A digit is a small image; a classifier maps it to 0–9.
- Today's pipeline (features $\rightarrow$ classifier) is exactly how classic OCR worked.
- Next week we replace hand features with a **CNN** that learns them — and accuracy jumps.

Toward learned features (Weeks 13–14)



- A CNN replaces HOG with filters it *learns* — the orange boxes are trained, not designed.
- Same pipeline shape as today; the difference is *who* designs the features.

Wrap-up & deliverable

What you should now be able to do

- Build a features $\rightarrow$ classifier pipeline (HOG + SVM/k-NN) on real images.
- Split train/test correctly and read a confusion matrix.
- Recognize overfitting (train $\gg$ test) and name cures.
- Explain why accuracy alone misleads, and when precision/recall matter.
- Use cross-validation and grid search without leaking the test set.

This week's deliverable

Due before Week 13 (see `project/`)

1. Extract HOG features for 2–3 classes from `E:/data`; build X, y.
2. Train k-NN and SVM with a stratified train/test split; report both accuracies.
3. Show the confusion matrix; identify which classes get confused.
4. Diagnose overfitting and try one cure (lower C, PCA, or more data); report whether the test score improved.

Features make it learnable; the split keeps you honest.

Hand-designed features hit a ceiling — next week we learn the features themselves.